

Przetwarzanie danych - indeksowanie hierarchiczne, przekształcanie ramek danych

Paweł Gliwny

Indeksowanie hierarchiczne

-Jest ważnym elementem biblioteki pandas **umożliwiającym przypisanie do jednej osi wielu poziomów indeksowania** (przypisanie dwóch lub więcej indeksów)

- Ta dość abstrakcyjna funkcja **umożliwia pracę z danymi o większej liczbie wymiarów** w formie charakteryzującej się mniejszą liczbą wymiarów
- Wyświetloną czytelnie serią, której indeksem jest obiekt *MultiIndex* .
- Obiekty o indeksie hierarchicznym obsługują tzw. indeksowanie częściowe. Indeksowanie to **umożliwia zwięzły wybór podzbioru danych**

Po co indeksowanie hierarchiczne?

Wyobraźmy sobie dane o sprzedaży w dwóch miastach, w latach 2023 i 2024:

miasto	rok	sprzedaż
Łódź	2023	120
Łódź	2024	145
Kraków	2023	200
Kraków	2024	230

- Dane mają **naturalną strukturę dwuwymiarową** (miasto $\times$ rok), ale chcemy je trzymać w **jednej** serii lub kolumnie.
- `MultiIndex` pozwala zakodować tę hierarchię w indeksie, zamiast tworzyć sztucznie szeroką tabelę.

Dane sprzedażowe w zwykłej ramce:

```
df = pd.DataFrame({  
    "miasto":    ["Łódź", "Łódź", "Kraków", "Kraków"],  
    "rok":       [2023,  2024,  2023,  2024],  
    "sprzedaz": [120,   145,   200,   230]  
})
```

	miasto	rok	sprzedaz
0	Łódź	2023	120
1	Łódź	2024	145
2	Kraków	2023	200
3	Kraków	2024	230

Chcemy sprzedaż w Łodzi w 2024:

```
df[(df.miasto == "Łódź") & (df.rok == 2024)][ "sprzedaz"]
```

- Za każdym razem ta sama, długa formuła.
- Trzy operacje: dwa filtry + wybór kolumny.
- Dla każdej nowej kombinacji piszemy to od nowa.

Rozwiązanie indeksowanie hierarchiczne (ang. *MultiIndex*)

```
sprzedaz = pd.Series(  
    [120, 145, 200, 230],  
    index=[["Łódź", "Łódź", "Kraków", "Kraków"],  
           [2023, 2024, 2023, 2024]]  
)  
print(sprzedaz)
```

```
Łódź    2023    120  
        2024    145  
Kraków  2023    200  
        2024    230  
dtype: int64
```

- Zewnętrzny poziom (miasto) jest wyświetlany **raz**, bez powtórzeń
 - to sygnał wizualny, że mamy **hierarchię**.
- `sprzedaz.index` zwraca obiekt `MultiIndex`.

```
sprzedaz.index
```

```
MultiIndex([( 'Łódź', 2023),  
            ( 'Łódź', 2024),  
            ('Kraków', 2023),  
            ('Kraków', 2024)],  
           )
```

Indeksowanie częściowe - w tym cała siła

Raz utworzony MultiIndex pozwala wybierać dane **na różnych poziomach**:

```
sprzedaz["Łódź"]          # wszystkie lata dla Łodzi  
# 2023      120  
# 2024      145
```

```
sprzedaz["Łódź", 2024]    # konkretna wartość  
# 145
```

```
sprzedaz.loc[:, 2023]     # wszystkie miasta, rok 2023  
# Łódź      120  
# Kraków    200
```

- **Zewnętrzny** poziom wybieramy zwykłym [...].
- **Wewnętrzny** poziom - przez .loc[:, wartość] (dwukropek = „wszystkie”).
- To jak **slicing** tablicy 2D, tylko że dane są logicznie 1D.

Bez i z *MultiIndex*

Porównajmy ten sam wybór:

Bez MultiIndex:

```
df[(df.miasto == "Łódź")  
    & (df.rok == 2024)]["sprzedaz"]
```

- Ta sama wartość.
- Jeden zapis zamiast trzech operacji.

Z MultiIndex:

```
sprzedaz["Łódź", 2024]
```

Analogia: indeks jak ścieżka do pliku

Pomyślmy o danych jak o **systemie katalogów**:

```
sprzedaz/  
├── Łódź/  
│   ├── 2023 → 120  
│   └── 2024 → 145  
└── Kraków/  
    ├── 2023 → 200  
    └── 2024 → 230
```

- Pojedyncza wartość ma **ścieżkę**, nie jedną etykietę: Łódź/2024.
- Zewnętrzny poziom (miasto) to folder, wewnętrzny (rok) to plik.
- MultiIndex to dokładnie ten pomysł przeniesiony na pandas: indeks jest **krotką**, a nie pojedynczą wartością.

Zmiana kształtu danych: stack, unstack, pivot

Te same dane można ułożyć **wąsko** lub **szeroko** — zmieniamy układ, nie wartości:

Wąska (long)				Szeroka (wide)		
Łódź	2023	120	→ unstack		2023	2024
	2024	145	← stack	Łódź	120	145
Kraków	2023	200		Kraków	200	230
	2024	230				

- unstack — poziom indeksu → kolumny (*potrzebuje MultiIndex*)
- stack — kolumny → poziom indeksu (*operacja odwrotna*)
- pivot_table — to samo jednym poleceniem + automatyczna agregacja

unstack - „wypchnij” poziom do kolumn

```
tabela = sprzedaz.unstack()  
tabela
```

rok	2023	2024
Łódź	120	145
Kraków	200	230

Łódź/2023, Łódź/2024 → Łódź: [2023]
[2024]

Kraków/2023, Kraków/2024 → Kraków:
[2023] [2024]

Wewnętrzny poziom indeksu staje się
kolumnami:

- Była **jedna kolumna** (Series) z 4 wierszami.
- Teraz **dwie kolumny** (DataFrame) z 2 wierszami.
- Analogia katalogów: rozłożyliśmy **zawartość folderów** obok siebie.

unstack - wybór poziomu

Domyślnie `unstack()` wypycha **ostatni** (wewnętrzny) poziom.

Możemy wybrać inny:

```
sprzedaz.unstack(level=0) # wypchnij zewnętrzny: miasto
```

	Łódź	Kraków
2023	120	200
2024	145	230

- Tabela jest **transponowana** - te same dane, inna perspektywa.

stack - operacja odwrotna

Kolumny wracają do indeksu - „składamy” tabelę z powrotem:

```
tabela.stack()
```

```
Łódź      2023      120  
          2024      145  
Kraków    2023      200  
          2024      230  
dtype: int64
```

- `unstack()` → z wąskiej do szerokiej (poziom indeksu → kolumny)
- `stack()` → z szerokiej do wąskiej (kolumny → poziomy indeks)

Praktyczne zastosowanie: set_index + unstack

Wracamy do naszej **płaskiej** ramki i przekształcamy ją w tabelę:

```
df.set_index(["miasto", "rok"])["sprzedaz"].unstack()
```

rok	2023	2024
miasto		
Łódź	120	145
Kraków	200	230

To samo co `pd.pivot_table` - ale budowane **krok po kroku**:

Kroki do stworzenie tabeli przestawnej (ang. *pivot*)

1. `set_index(["miasto", "rok"])` → MultiIndex na wierszach,
 2. `["sprzedaz"]` → wyciągnij jedną kolumnę (Series),
 3. `unstack()` → rok przechodzi do kolumn,
- Ten wzorzec `set_index` → `unstack` to **ręczna tabela przestawna**.
 - Warto go rozumieć, zanim użyjemy `pivot_table`.

pivot_table - to samo, jedno polecenie

```
pd.pivot_table(df,  
    index="miasto",      # co w wierszach?  
    columns="rok",       # co w kolumnach?  
    values="sprzedaz",   # co w komórkach?  
    aggfunc="sum"        # jak agregować?  
)
```

rok	2023	2024
miasto		
Łódź	120	145
Kraków	200	230

Po co agregacja?

Wyobraźmy sobie dane **kwartalne** zamiast rocznych:

```
df2 = pd.DataFrame({
    "miasto": ["Łódź"]*4 + ["Kraków"]*4,
    "rok":     [2023]*4 + [2023]*4,
    "kwartał": ["Q1", "Q2", "Q3", "Q4"] * 2,
    "sprzedaz": [30, 25, 35, 30, 50, 45, 55, 50]
})
pd.pivot_table(df2,
    index="miasto", columns="rok",
    values="sprzedaz", aggfunc="sum")
```

```
rok      2023
miasto
Kraków   200
Łódź     120
```

- Łódź/2023 ma **4 wiersze** (Q1–Q4) → `pivot_table` sumuje je automatycznie.
- `unstack` w tej sytuacji rzuciłby błąd (duplikaty w indeksie).
- Domyślna `aggfunc` to "mean" — zwykle chcemy ją podać jawnie.

pivot_table - podsumowanie

Trzy pytania → trzy argumenty:

Pytanie	Argument	Przykład
Co w wierszach?	index	"miasto"
Co w kolumnach?	columns	"rok"
Co w komórkach?	values + aggfunc	"sprzedaz", "sum"

unstack

- Dane już mają MultiIndex
- Unikalne kombinacje
- Pełna kontrola krok po kroku

pivot_table

- Dane w „płaskiej” ramce
- Mogą być duplikaty
- Szybkie podsumowanie jednym poleceniem

Agregacja po poziomach indeksu

Mając MultiIndex, możemy agregować po **wybranym poziomie** bez unstack:

```
sprzedaz.index.names = ["miasto", "rok"]  
sprzedaz.groupby(level="miasto").sum()
```

```
miasto  
Łódź      265  
Kraków    430  
dtype: int64
```

```
sprzedaz.groupby(level="rok").mean()
```

```
rok  
2023    160.0  
2024    187.5  
dtype: float64
```

- `level="miasto"` → „sklej lata, pokaż wynik per miasto”
- `level="rok"` → „sklej miasta, pokaż wynik per rok”

Agregacja w DataFrame z MultiIndex

```
frame = pd.DataFrame({  
    "sprzedaz": [120, 145, 200, 230],  
    "koszty":   [80,  90, 130, 140]  
}, index=[["Łódź", "Łódź", "Kraków", "Kraków"],  
         [2023,  2024,  2023,   2024]])  
frame.index.names = ["miasto", "rok"]  
frame.groupby(level="miasto").sum()
```

	sprzedaz	koszty
miasto		
Kraków	430	270
Łódź	265	170

- Agregacja działa na **wszystkich kolumnach** jednocześnie.
- Można wybrać jedną: `frame["sprzedaz"].groupby(level="rok").max()`

Dostępne funkcje: `sum`, `mean`, `min`, `max`, `count`, `std`, ...

Wiele funkcji naraz - agg

```
frame.groupby(level="miasto").agg(["sum", "mean"])
```

	sprzedaz		koszty	
	sum	mean	sum	mean
miasto				
Kraków	430	215.0	270	135.0
Łódź	265	132.5	170	85.0

- Wynik ma **hierarchiczne kolumny** (MultiIndex na osi kolumn).
- Dostęp: wynik["sprzedaz", "sum"] lub wynik["sprzedaz"]["sum"]

Zdefiniowanie własnej funkcji

```
frame["sprzedaz"].groupby(level="miasto").agg(  
    total="sum", srednia="mean", rozstep=lambda x: x.max()-x.min()  
)
```

	total	srednia	rozstep
miasto			
Kraków	430	215.0	30
Łódź	265	132.5	25